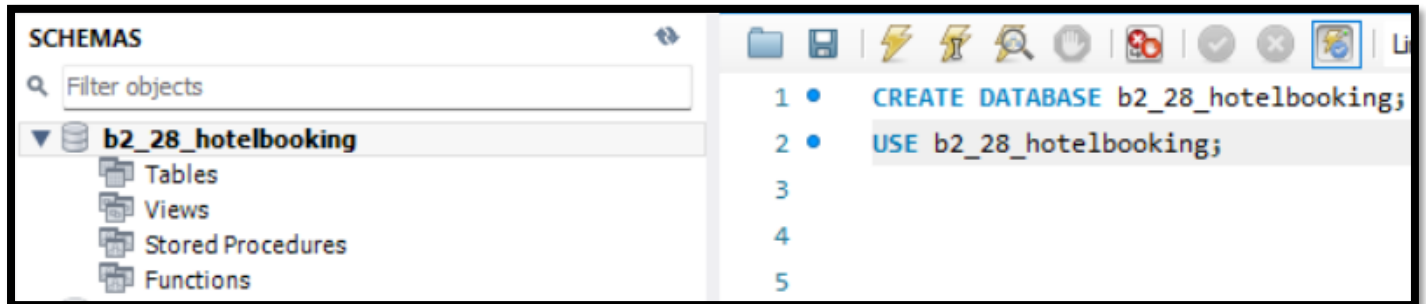


Experiment 3 :

• CREATE Database:

Creates a new database to store tables and data.



• CREATE Table:

Creates a new table with specified columns and constraints.

❖ Unique:

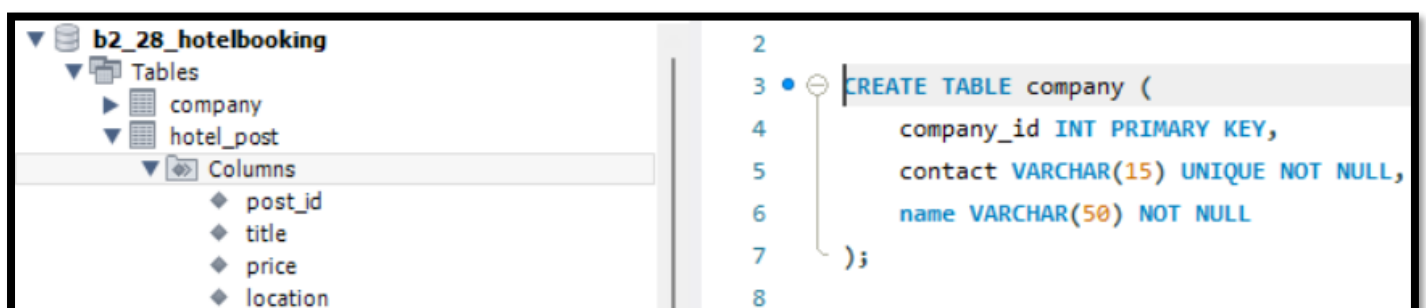
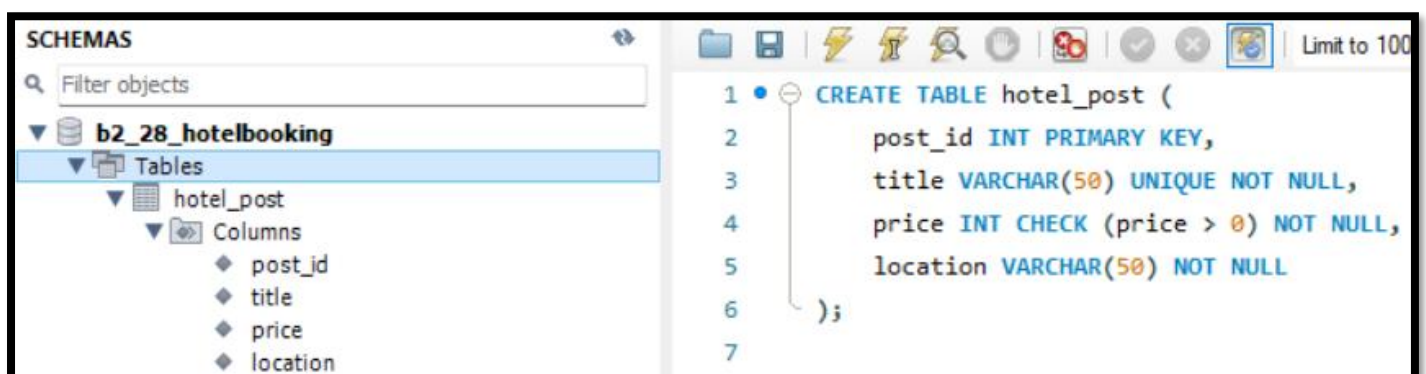
Ensures all values in a column are different.

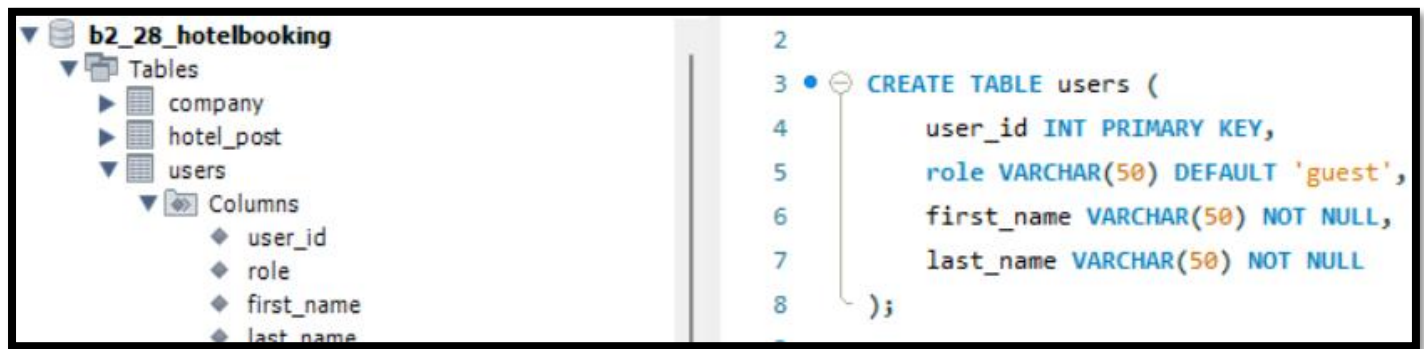
❖ PRIMARY KEY:

Uniquely identifies each record in a table (cannot be NULL or duplicate).

❖ DEFAULT:

Automatically assigns a value if no value is provided during insert.





```

2
3 CREATE TABLE users (
4     user_id INT PRIMARY KEY,
5     role VARCHAR(50) DEFAULT 'guest',
6     first_name VARCHAR(50) NOT NULL,
7     last_name VARCHAR(50) NOT NULL
8 );

```

❖ FOREIGN KEY:

Creates a relationship between two tables by referencing another table's primary key.

❖ CHECK:

Restricts values based on a condition.



```

2
3 CREATE TABLE contact (
4     user_id INT,
5     contact VARCHAR(15),
6     PRIMARY KEY (user_id, contact),
7     FOREIGN KEY (user_id) REFERENCES users(user_id)
8 );
9

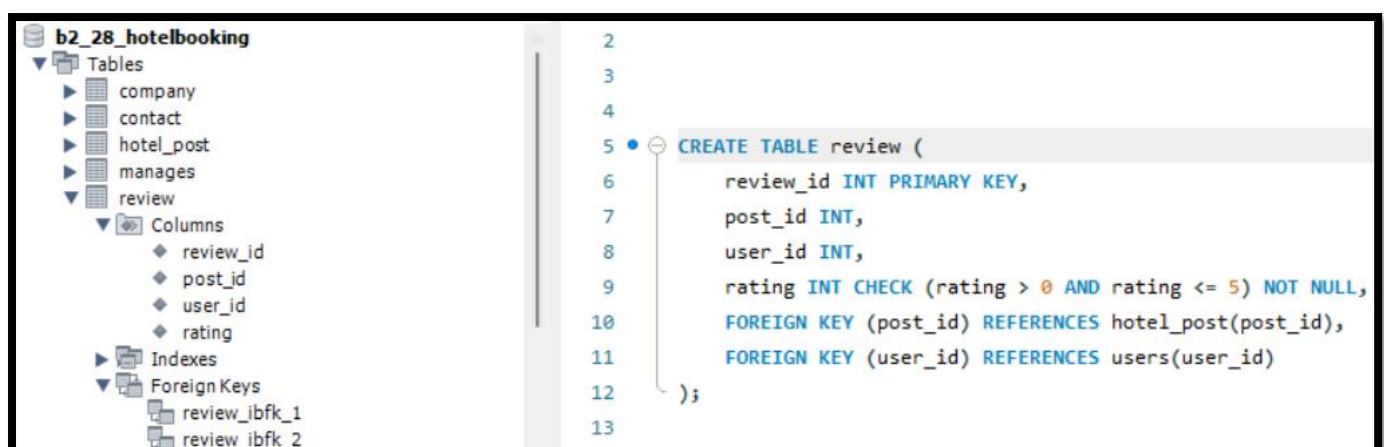
```



```

2
3
4 CREATE TABLE manages (
5     company_id INT,
6     post_id INT,
7     PRIMARY KEY (company_id, post_id),
8     FOREIGN KEY (company_id) REFERENCES company(company_id),
9     FOREIGN KEY (post_id) REFERENCES hotel_post(post_id)
10 );
11

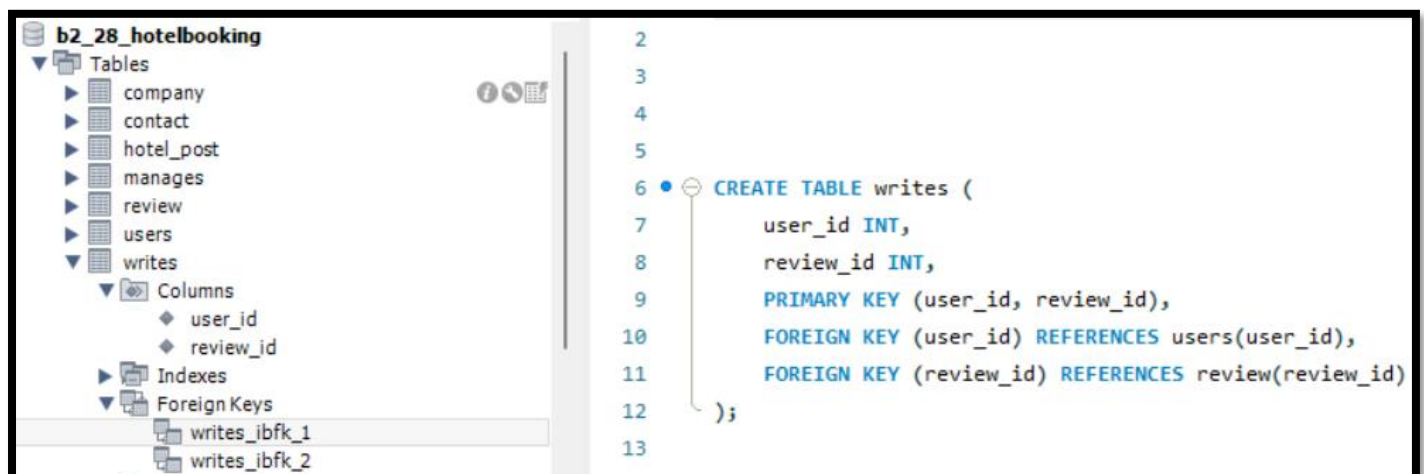
```



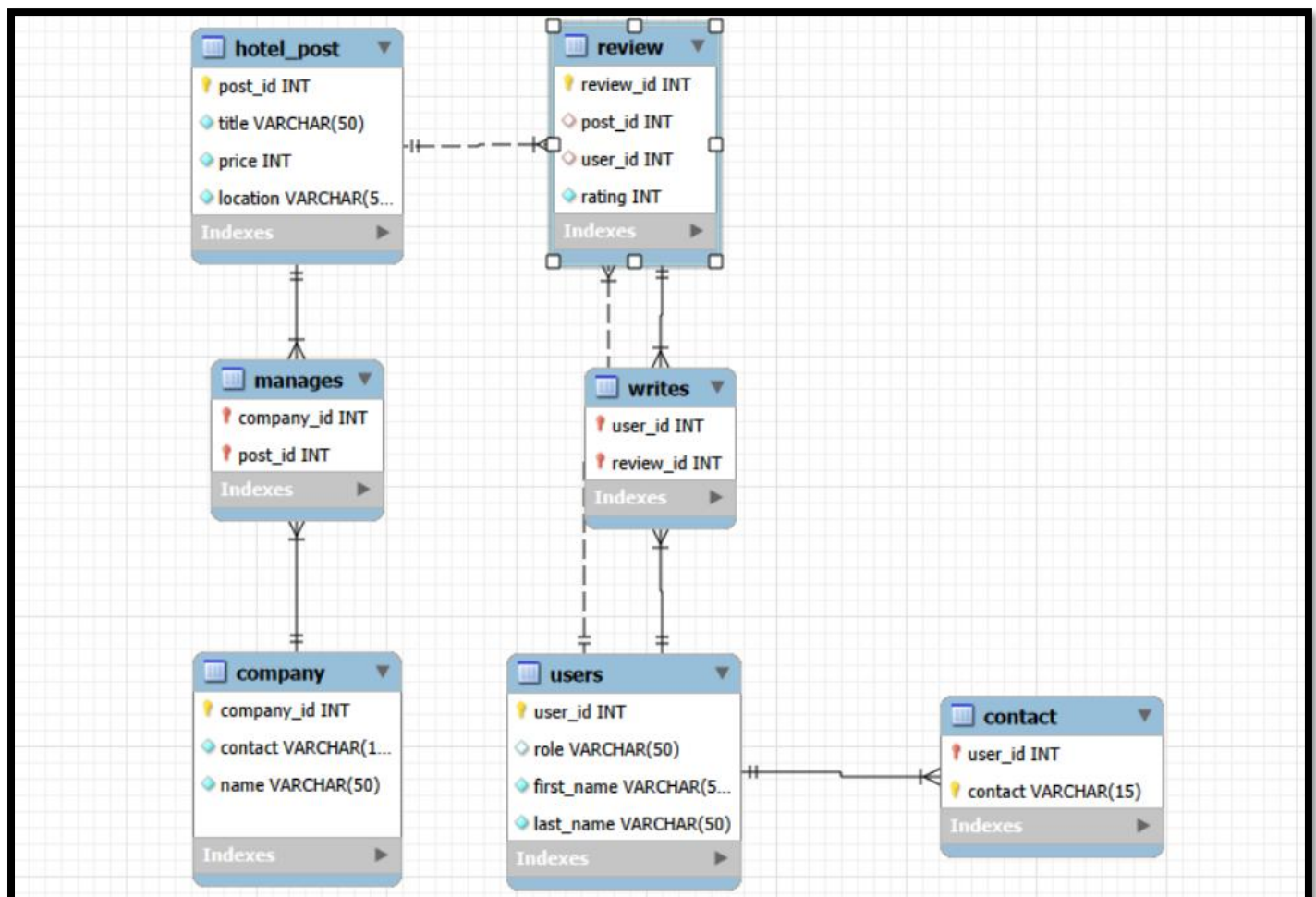
```

2
3
4
5 CREATE TABLE review (
6     review_id INT PRIMARY KEY,
7     post_id INT,
8     user_id INT,
9     rating INT CHECK (rating > 0 AND rating <= 5) NOT NULL,
10    FOREIGN KEY (post_id) REFERENCES hotel_post(post_id),
11    FOREIGN KEY (user_id) REFERENCES users(user_id)
12 );
13

```

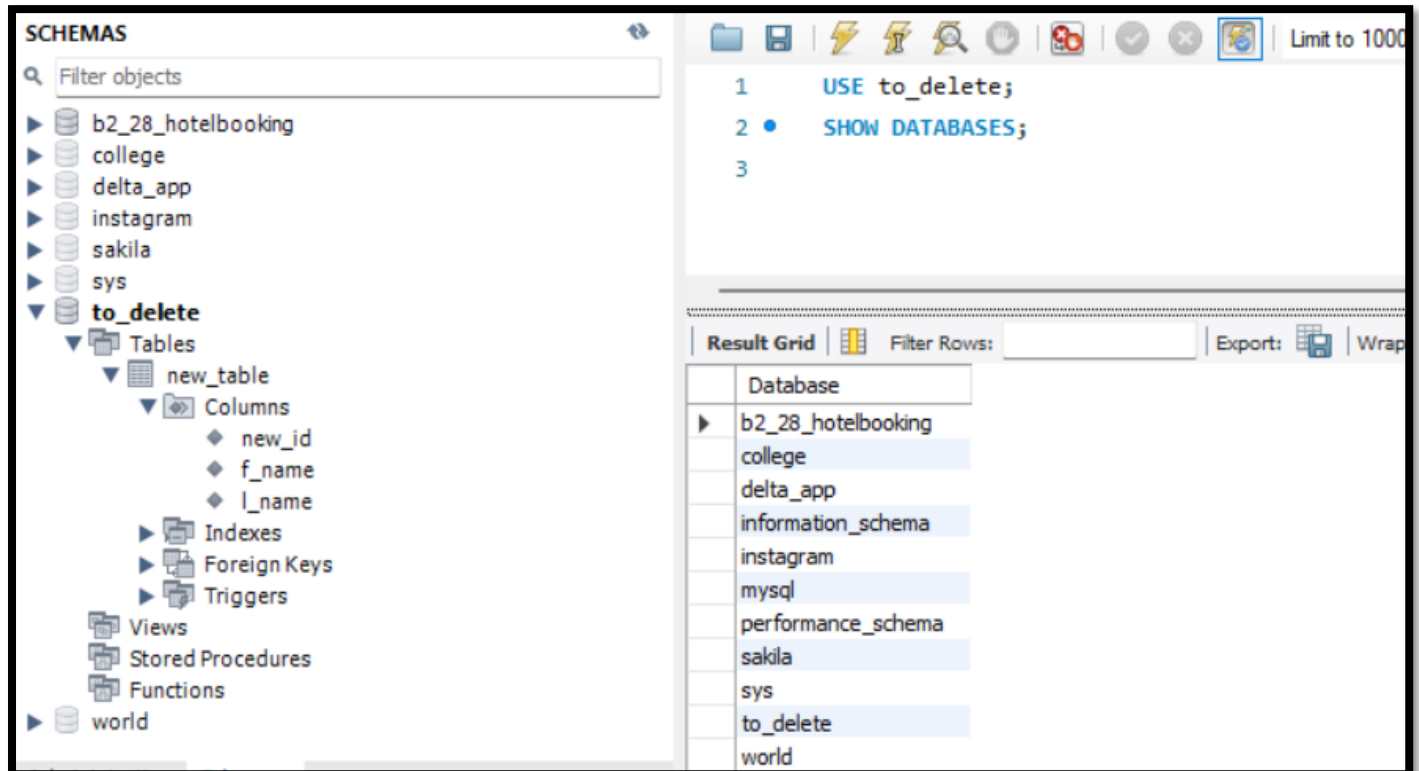


• EER diagram via Reverse Engineering Database:



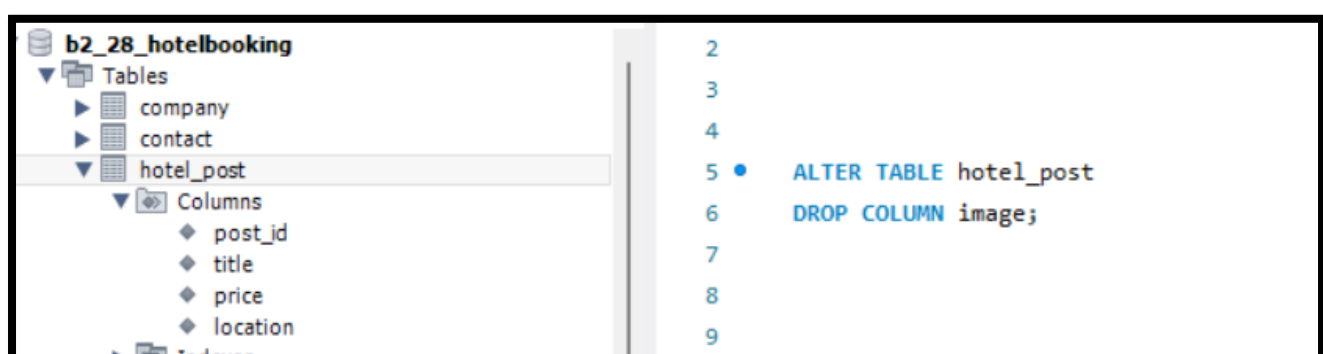
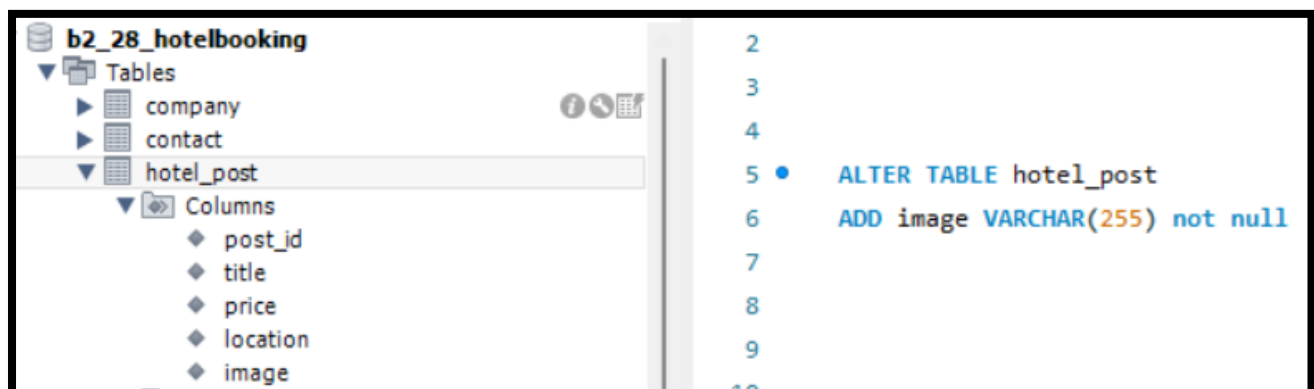
- **SHOW Database:**

Displays all databases available in the MySQL server.



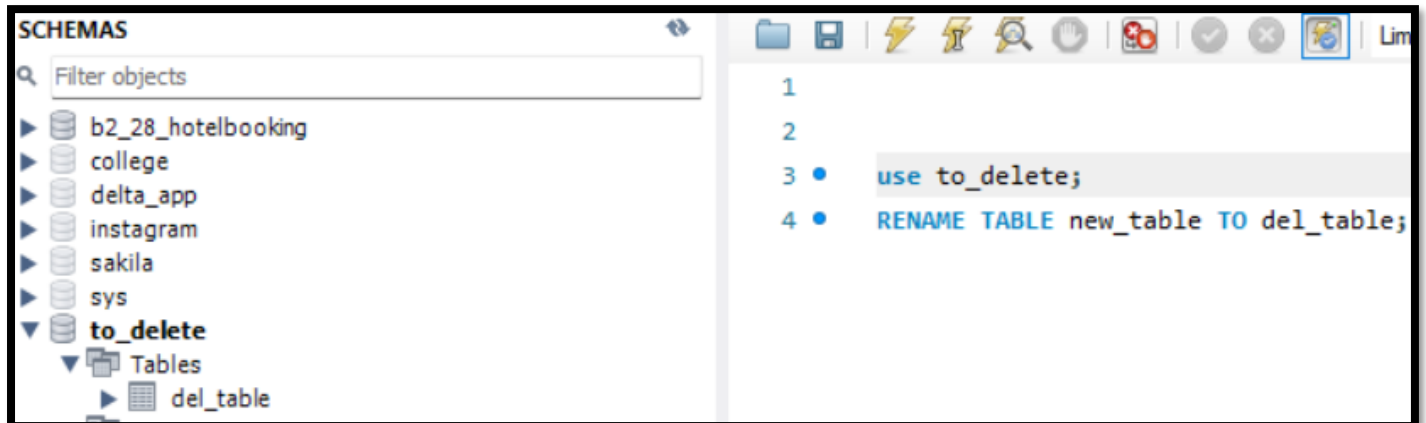
- **ALTER Table:**

Modifies the structure of an existing table.



- **RENAME Table:**

Changes the name of an existing table.



- **DROP Table:**

Deletes the entire table permanently.

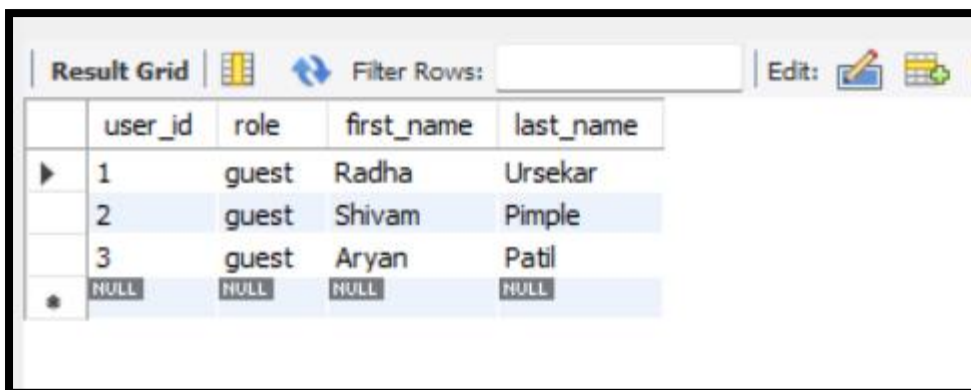


- **DROP Database:**

Deletes the entire database permanently.




```
1 • INSERT INTO users (user_id, first_name, last_name)
2 VALUES
3 (1, 'Radha', 'Ursekar'),
4 (2, 'Shivam', 'Pimple'),
5 (3, 'Aryan', 'Patil');
6
7 • DELETE FROM users
8 WHERE user_id = 3;
9
10 • TRUNCATE TABLE users;
```

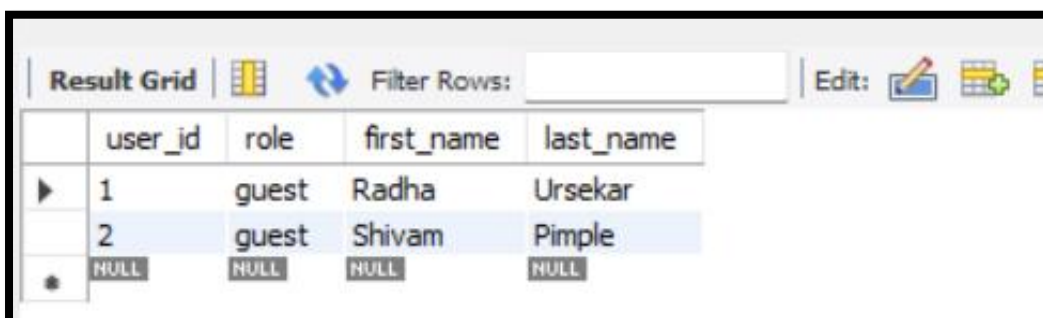


The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with columns: user_id, role, first_name, and last_name. There are three rows of data, all with the role 'guest'. The first row has user_id 1, first_name 'Radha', and last_name 'Ursekar'. The second row has user_id 2, first_name 'Shivam', and last_name 'Pimple'. The third row has user_id 3, first_name 'Aryan', and last_name 'Patil'. Below the data rows is a row with NULL values for all columns, preceded by an asterisk icon.

	user_id	role	first_name	last_name
▶	1	guest	Radha	Ursekar
	2	guest	Shivam	Pimple
	3	guest	Aryan	Patil
*	NULL	NULL	NULL	NULL

- **DELETE FROM Table:**

Removes selected rows from a table based on a condition.

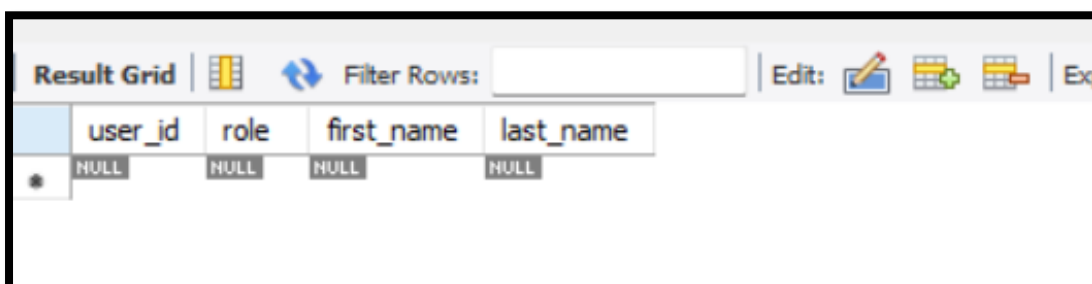


The screenshot shows the same database interface as before, but now only two rows of data are present. The row with user_id 3 has been removed. The first row has user_id 1, first_name 'Radha', and last_name 'Ursekar'. The second row has user_id 2, first_name 'Shivam', and last_name 'Pimple'. The NULL row remains at the bottom.

	user_id	role	first_name	last_name
▶	1	guest	Radha	Ursekar
	2	guest	Shivam	Pimple
*	NULL	NULL	NULL	NULL

- **TRUNCATE Table:**

Removes all rows from a table quickly while keeping its structure.



The screenshot shows the database interface after truncating the table. The table structure is preserved, but all data rows have been removed. Only the NULL row is visible at the bottom of the grid.

	user_id	role	first_name	last_name
*	NULL	NULL	NULL	NULL

